Typescript

It's JavaScript with types.

Yeah, JavaScript hi hey bs ismay apko static typing ya type safety milti hey jisay apke code behavior predictable rehta hey.

Yeah, aik addon hey JavaScript k uper jo hame static typing or kuxh extra feature provide karti hey but eventually type script JS mey hi transpile hoti hey, yeah direct run nahi hoti.

Type Annotation

Type annotation ka matlab hai ke developer **`khud se`** batata hai ke kisi variable, function parameter, ya object ki type kya hogi. Is mein hum variable ke naam ke baad colon (`:`) aur phir type likhte hain.

```
let username: string = "Zeeshan";
let age: number = 25;
let isStudent: boolean = true;

// Function mein annotation
function add(a: number, b: number): number {
    return a + b;
}
```

Type Inference

Type inference TypeScript ki ek **`smart feature`** hai. Agar aap khud se type nahi batate, to TypeScript value ko dekh kar **`khud hi andaza (infer)`** laga leta hai ke uski type kya honi chahiye.

```
// TypeScript ne khud hi ise 'number' maan liya
let score = 100;

// Yeh error dega kyunke type 'number' infer ho chuki hai
score = "Win";
```

Type Any

Any ko Escape Hatch bhi boltey heyn. `any` ka matlab hai ke aap TypeScript ko keh rahe hain ke "is variable ki type check mat karo." Ye bilkul waisa hi hai jaise simple JavaScript likhna.

```
let meraData: any = 10;
meraData = "Ab mein string hoon"; // Koi error nahi
meraData.kuchBhi(); // Runtime par crash ho sakta hai,
```

Usage: Jab aapko pata na ho ke data kahan se aa raha hai ya data ki type badalti rahegi.

Khatra: Is se TypeScript ka asal maqsad (type safety) khatam ho jata hai kyunke ye kisi bhi kism ka error nahi deta.

Type Unknown

TypeScript mein `unknown` type ko **"Safe Any"** kaha jata hai. Yeh `any` ki tarah hi kaam karta hai kyunke aap is mein kisi bhi qism ki value (string, number, object, etc.) store kar sakte hain, lekin yeh `any` se zyada sakht (strict) hai.

Jab aap kisi variable ko `unknown` assign karte hain, to TypeScript aapko us variable ko tab tak use nahi karne deta jab tak aap uski **Type Guarding** ya **Type Narrowing** na kar lein.

Agar hum `any` use karein:

```
let value: any = "Hello";

// Chale ga (lekin error prone hai agar value number hui)
console.log(value.toUpperCase());
```

Agar hum `unknown` use karein:

```
let value: unknown = "Hello";

// Error: 'value' is of type 'unknown'.
console.log(value.toUpperCase());

// Ab chale ga (Type narrowing ho gayi)
if (typeof value === "string") {
    console.log(value.toUpperCase());
}
```

Type Literal

ismay ham bajye yeah batanay k value number type ki hogi ya string type ki koee bhi value hogi iskay bajai ham btatatay heyn k is type ki konsi specific value hogi.

Yani ham bata deytay heyn k yeah variable literally ya exactly isi specific value k equal hoga.

```
let status: "success";

status = "success" // valid
status = "pending" // error
```

Type Union

Union types aapko ijazat dete hain ke ek variable ko ek se zyada types assign karein. Is ke liye pipe symbol `|` use hota hai.

```
let subscribers: string | number;
subscribers = 10,000;      // Valid
subscribers = "1M+"; // Valid
subscribers = true;  // Error! Kyunke boolean union mein nahi hai
```

Union sirf `string` ya `number` tak mehdood nahi, aap specific values bhi set kar sakte hain:

```
type Status = "success" | "error" | "loading";

let currentStatus: Status = "success";
currentStatus = "pending"; // Error! Kyunke ye list mein nahi hai
```

Type Array

Yeah array or uski values kis type hogi yeah cheez specify karnay kelye use hoti hey.

```
let fruites: string[] = ["🍇", "🍌"]
```

The term `string` indicates that the data array will hold strings, and the brackets indicate that it's an array type.

```
let fullName: [string, string] = ["Asif", "Shahzad"];
```

Uper wali example mey full name array hey jismay sirf do hi value allowed hongi or dono k type string hogi.

Type Alias or Custom Types

Type Alias ka maqsad **`Code Reusability`** aur **`Readability`** hai. Jab humein ek hi type (ya combination of types) baar baar use karni ho, to hum `type` keyword ka istemal karte hain.

Ye bilkul waisa hi hai jese kisi lambay naam ko ek "nickname" dey dena taake baar baar mehnat na karni paray.

1. Simple Union Type Alias

Jaisa jab ek variable ek se zyada types accept kare:

```
type ID = string | number;

function getOrder(orderId: ID) {
    console.log("Fetching order: " + orderId);
}
```

2. Object Type Alias (Bohat Aham)

Sirf simple types hi nahi, aap poore **`Objects`** ka structure bhi define kar sakte hain. Is se code saaf rehta hai.

```
type User = {
    id: ID; // Hum ne purana alias yahan use kar liya!
    username: string;
    email: string;
    isActive: boolean;
};

const newUser: User = {
    id: 101,
    username: "ali_khan",
    email: "ali@example.com",
    isActive: true
};
```

3. Function Type Alias

Aap functions ke signature ke liye bhi alias bana sakte hain.

```
type MathOp = (a: number, b: number) => number;

const add: MathOp = (x, y) => x + y;
const multiply: MathOp = (x, y) => x * y;
```

Type Alias ke Faide (Summary)

`DRY Principle:` "Don't Repeat Yourself" – Ek dafa likho aur har jagah use karo.

`Easy Maintenance:` Agar kal ko `ID` sirf `string` karni paray, to aap ko 50 jagah change nahi karna parega, sirf alias main change karen ge.

`Meaningful Names:` `string | number` se behtar `OrderID` ya `UserID` lagta hai, jis se code parhne wale ko samajh aata hai ke ye kya cheez hai.

Type Alias vs Interface

`Type Interface` sirf object k structure ko define karney kelye use hota hey, or yeah extendable hey, yeah OOP principal k sath ziyada compaitable hey.

Recommended to use the interface for objects.

```
interface User {
    name: string
    age: number
}

interface Admin extends User{
    dashboard: true
}
```

The Admin also has the properties definition from the User.

`Interface Type` ko ager app dobara same name say declare kartey ho to don objects k structure mey di properties merge ho jati heyn.

```
interface Windows {
    name: string
}

interface Windows{
    status: string
}

let wind: Windows = {name: "Chrome", status: "active"};(src, {})
```

`Type Alias` primitive data types kelye best hey but yeah extendable in case of objects but ap iskay sath bhi alias for objects create kar saktey heyn.

```
type User = {
  name: string;
  age: number;
};

type Admin = User & {
  dashboard: boolean;
};
```

`Type Object Alias` ko dobara declare karney per error ata hey

```
type Windows {
  name: string;
}

type Windows { // Duplicate identifier 'Windows'.
  status: string;
}
```

Type Narrowing

TypeScript code ko analyze karta hai aur check karta hai ke kisi specific point par variable ki type kya ho sakti hai. Jab aap `if` ya `else` conditions lagate hain, to TypeScript automatically type ko "narrow" (chota) kar deta hai.

```
function printId(id: string | number) {
  if (typeof id === "string") {
    // Is block ke andar 'id' sirf string hai
    console.log(id.toUpperCase());
  } else {
    // Yahan 'id' sirf number hai
    console.log(id.toFixed(2));
  }
}
```

Type Assertion

`Type Assertion` ka matlab hai compiler ko yeh batana ke *"Mujhe pata hai main kya kar raha hoon, is variable ki type wahi hai jo main keh raha hoon.*

Asal mein, kabhi kabhi aapko variable ki type ke baare mein TypeScript se zyada maloomat hoti hai (maslan API se response aate waqt). Type assertion ke zariye aap TypeScript ke default inference ko override kar dete hain.

`Note:` Type assertion se data convert nahi hota (casting nahi hoti), yeh sirf compiler ko chup karwane aur code-completion behtar karne ke liye hota hai.

Type assertion likhne ke do main tareeqay hain:

Type Assertion using `as` and `<type>`

TypeScript ko nahi pata ke `response` string hai, isliye length nahi nikalne de

```
let response: unknown = "Yeh ek string hai";
let length = someValue.length; // ❌ Error
```

But ham type assertion k zariye compiler ko yeah bata saktey heyn k eventually mujeh pata lag giya hey response string type ka hey to tum bhi yahi ma lo

```
let length = (response as string).length
```

isi ko karnay ka dosra method yeah bhi hey

```
let length = (<string>response).length
```

`Note:` React/JSX mein angle-bracket use nahi hota kyunke wahan confuse ho jata hai, isliye `as` behtar hai.

Optional Parameter

TypeScript mein by-default har parameter zaroori (required) hota hai. Agar aap koi parameter chhor dena chahte hain, to uske naam ke saath `?` lagate hain.

```
function greet(name: string, message?: string) {
    if (message) {
        return `Hello ${name}, ${message}`;
    } else {
        return `Hello ${name}`;
    }
}
// Sirf ek argument: Output "Hello Zeeshan"
console.log(greet("Zeeshan"));

// Do arguments: Output "Hello Zeeshan, Ao Chai Piyo"
console.log(greet("Zeeshan", "Ao Chai Piyo"));
```

`Undefined Value:` Agar aap optional parameter ki value nahi dete, to TypeScript usay `undefined` maanta hai.

`Position:` Optional parameters hamesha **`last`** mein aane chahiye. Aap required parameter se pehle optional nahi rakh sakte.

Type Guard

Farz karein ek variable ki type `string | number` (Union Type) hai. Agar aap us par `.toUpperCase()` call karenge, to TypeScript error dega kyunke ho sakta hai us waqt variable mein `number` ho. Type guards is maslay ko hal karte hain.

`typeof` Guard

Yeh basic types (string, number, boolean, etc.) ko check karne ke liye use hota hai.

```
function printId(id: string | number) {
    if (typeof id === "string") {
        // Yahan TypeScript ko pata hai ke 'id' string hai
        console.log(id.toUpperCase());
    } else {
        // Yahan 'id' lazmi number hoga
        console.log(id.toFixed(2));
    }
}
```

`instanceof` Guard

Yeh check karne ke liye hota hai ke koi object kisi specific **`Class`** ka instance hai ya nahi.

```
class Dog { bark() { console.log("Woof!"); } }
class Cat { meow() { console.log("Meow!"); } }

function makeNoise(animal: Dog | Cat) {
    if (animal instanceof Dog) {
        animal.bark(); // TypeScript confirms it's a Dog
    } else {
        animal.meow(); // TypeScript confirms it's a Cat
    }
}
```

`in` Operator Guard

Yeh check karta hai ke kisi object ke andar koi specific **`property`** maujood hai ya nahi.

```
interface Car { drive: () => void }
interface Boat { sail: () => void }

function move(vehicle: Car | Boat) {
    if ("drive" in vehicle) {
        vehicle.drive();
    } else {
        vehicle.sail();
    }
}
```

Custom Type Guard (Type Predicates)

PENDING...

Type Never

TypeScript mein `never` type aik aisi type hai jo un values ko represent karti hai jo **`kabhi nahi ho saktien`** (values that will never occur).

Isay aksar "Impossible type" bhi kaha jata hai.

Aisa Function jo Kabhi Khatam Na Ho

Agar koi function hamesha error throw kare ya "infinite loop" mein chala jaye, to uska return type `never` hota hai kyunke wo kabhi koi value return nahi kar pata.

```
function throwError(message: string): never {
    throw new Error(message);
}

function keepLooping(): never {
    while (true) {
        console.log("Chalta hi ja raha hoon...");
    }
}
```

Exhaustive Checking (Logic Safety)

Type Guards use karte waqt, agar aapne tamam types ko handle kar liya ho, to aakhri bachi hui condition `never` ban jati hai.

```ts
type Status = "pending" | "deliverd";

function checkStatus(status: Status) {
  if (status === "pending") {
    console.log("Status is pending");
  } else if (status === "deliverd") {
    console.log("Status is deliverd");
  } else {
    // Is 'else' block ke andar agar aap 'status' par mouse rakhenge
    // to wahan 'never' likha aayega.
    console.log(status);
  }
}
```